

00_download_dataset

September 27, 2026

1 Download Audio MNIST Dataset

Downloads the [Audio MNIST](#) dataset from Hugging Face and saves metadata to data/processed/.

```
[1]: from sdoml_task1.config import PROJECT_DIR, DATA_DIR, N_MFCC
from sdoml_task1.features import extract_features, decode_audio,
    ↪ build_feature_dataset
from sdoml_task1.dataset import AudioMNISTFeaturesDataset
from sdoml_task1.modeling.model import Net

from datasets import load_dataset, Audio
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

```
[2]: ds = load_dataset("gilkeyio/AudioMNIST")
ds = ds.cast_column("audio", Audio(decode=False))

print(f"Train: {len(ds['train'])} samples")
print(f"Test: {len(ds['test'])} samples")
print(f"Columns: {ds['train'].column_names}")
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Train: 24000 samples

Test: 6000 samples

Columns: ['speaker_id', 'audio', 'digit', 'gender', 'accent', 'age', 'native_speaker', 'origin']

```
[3]: records = [{
    "digit": s["digit"],
    "speaker_id": s["speaker_id"],
    "gender": "male" if s["gender"] == 0 else "female",
    "accent": s["accent"],
    "age": s["age"],
    "origin": s["origin"],
    "split": split_name,
} for split_name, split in ds.items() for s in split]
```

```

df = pd.DataFrame(records)
df.to_csv(DATA_DIR / "audio_mnist_metadata.csv", index=False)
print(f"Saved: {DATA_DIR / 'audio_mnist_metadata.csv'}")
print(f"Total: {len(df)} records")
df.head()

```

Saved: /home/amets/Documents/MASTER/SDOML/project/SDOML-project1/data/audio_mnist_metadata.csv
Total: 30000 records

```

[3]:   digit speaker_id gender accent age          origin split
0      7          59 female German  31.0 Europe, Germany, Berlin train
1      7          59 female German  31.0 Europe, Germany, Berlin train
2      2          59 female German  31.0 Europe, Germany, Berlin train
3      3          59 female German  31.0 Europe, Germany, Berlin train
4      9          59 female German  31.0 Europe, Germany, Berlin train

```

1.0.1 AUDIO MNIST ANALYSIS

Before creating the model, we first made a brief analysis of the dataset itself to check for possible biases. According to the dataset [source](#):

“The audioMNIST dataset has 50 English recordings per digit (0-9) of 60 speakers. There are 60 participants in total, with 12 being women and 48 being men, all featuring a diverse range of accents and country of origin. Their ages vary from 22 to 61 years old. This is a great dataset to explore a simple audio classification problem: either the digit or the gender.”

```

[4]: print(df.info())

```

```

<class 'pandas.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   digit           30000 non-null  int64
1   speaker_id      30000 non-null  str
2   gender          30000 non-null  str
3   accent          30000 non-null  str
4   age             29500 non-null  float64
5   origin          30000 non-null  str
6   split           30000 non-null  str
dtypes: float64(1), int64(1), str(5)
memory usage: 2.8 MB
None

```

```

[5]: digit_counts = df["digit"].value_counts().sort_index().reset_index()
digit_counts.columns = ["Digit", "Count"]
print(digit_counts)

```

	Digit	Count
0	0	3000
1	1	3000
2	2	3000
3	3	3000
4	4	3000
5	5	3000
6	6	3000
7	7	3000
8	8	3000
9	9	3000

Each number has the same amount of samples, so the dataset is well-distributed in this aspect.

After that, we checked the details from the speakers

```
[6]: df["age"].value_counts()
```

```
[6]: age
26.0    5000
27.0    3000
23.0    3000
25.0    3000
31.0    2500
24.0    2000
30.0    2000
28.0    2000
29.0    1500
22.0    1500
33.0    1000
61.0     500
32.0     500
35.0     500
41.0     500
34.0     500
36.0     500
Name: count, dtype: int64
```

1.1 Data visualization

In this cell we create all the graphs required to analyze the dataset. Rather than showing them here, we generate them and save them in `assets/`

```
[ ]: palette = sns.color_palette("Set2", n_colors=10)

plt.figure(figsize=(6, 5))

#Gender distribution
gender_count = df["gender"].value_counts(normalize=True)*100
```

```

gender_count.plot.bar(ax=plt.gca(), color=palette[:len(gender_count)])
plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Percentage")
plt.grid(axis="y", alpha=0.3)
plt.savefig("../assets/dataset_analysis/dataset_gender_distribution.png",
            dpi=150)
plt.close()

# Age Histogram
age_count = df["age"].value_counts(normalize=True)*100
df['age'].hist(ax=plt.gca(), bins=20, color=palette[0])
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.title('Distribution of Participants by age')
plt.grid(axis="y", alpha=0.3)
plt.grid(axis="x", alpha=0.0)
# plt.xaxis.grid(False)
plt.savefig("../assets/dataset_analysis/dataset_age_distribution.png", dpi=150)
plt.close()

#Accent distribution
accent_count = df["accent"].value_counts(normalize=True)*100
accent_count.plot.bar(ax=plt.gca(), color=palette[:len(accent_count)])
plt.title("Dataset Accent Distribution")
plt.xlabel("Accent")
plt.ylabel("Percentage")
plt.grid(axis="y", alpha=0.7)
plt.savefig("../assets/dataset_analysis/dataset_accent_distribution.png",
            dpi=150)
plt.close()

#Accent relative to gender
sns.boxplot(data=df, x="gender", y="age", hue="gender", palette="Set2",
            legend=False)
plt.title("Age relative to the gender")
plt.xlabel("Gender")
plt.ylabel("Age")
plt.savefig("../assets/dataset_analysis/dataset_age_gender.png", dpi=150)
plt.close()

```

There is a clear bias towards younger males with german accent. There is an underrepresentation of females, with only 20% of the dataset being female voices, and the representation of older voices is non-existent, with a couple of exceptions.

The gender groups show a very similar distribution, mainly clustered around the mid to late twenties, with only 2 exceptions for the male group.

For our dataset we transform the audio signals into 26 digit vectors to process the dataset and then we base our model on the pythorch example of the course, then adapt it to meet our needs (unit 2.2 slide 9)

[]: